

SIMATS ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – EXPERIMENTS

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO3 – Precision (BL3)	Assessment:	Experiments
Weightage:	45%	Total Marks:	100
CO3 Statement:	Design processor organization by constructing pipelined datapath structures, identifying hazard types (data, control, structural), and comparing superscalar techniques such as out-of-order execution, speculative execution, and simultaneous multithreading (SMT).		
Student Name:	Kolar Umme Sadiya	Reg. No.:	192425117
Section / Batch:	3 rd year /AI&ML	Date:	11/08/2026

Instructions to Students

- Identify the relevant concepts of register transfers, ALU operations, bus organization, pipelining, hazards, and superscalar processing.
- Analyze the execution of data transfers, arithmetic and logic operations, instruction processing, and parallel execution.
- Apply suitable techniques such as multiple buses, pipelining, stalling, forwarding, branch prediction, and speculative execution.
- Compute performance measures such as execution time, clock cycles, speedup, throughput, and resource utilization.
- Interpret the results by evaluating performance improvements, bottlenecks, and the suitability of each architectural technique.

QUESTIONS

1. Construct a pipeline scenario containing dependent instructions, branch instructions, and competing resource requirements, and apply techniques such as pipeline stalling, operand forwarding, branch prediction, and resource duplication to minimize performance losses.
2. Develop a computer system model using a hierarchical bus structure to transfer data among the CPU, memory, and I/O devices, and analyse how bus priority, bandwidth, and transfer conflicts affect overall system performance.
3. Develop a processor model that supports multiple instruction issue, out-of-order execution, and speculative execution, and analyse how instruction scheduling, execution units, and simultaneous multithreading affect instruction throughput and processor resource utilization.
4. Design and simulate a processor data path that performs register-to-register data transfers and arithmetic operations such as increment, decrement, addition, and subtraction, and analyse the control signals and execution sequence involved in each operation.
5. Simulate a pipelined processor executing a sequence of arithmetic, memory, and branch instructions, and calculate the pipeline execution time, throughput, and speedup by comparing the results with sequential instruction execution.

Code of Conduct

I Kolar Umme Sadiya-192425117 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Kolar Umme Sadiya

RUBRICS FOR CALCULATION

Criteria	Weightage(%)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Marks Evaluation:

Q. No	Understanding of Concepts(25)	Experimental Design (30)	Use of Tools(20)	Analysis of Results (15)	Documentation (10)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	/ 30	/ 20	/ 15	/ 10	/ 100

Course Coordinator

Faculty Incharge

EXPERIMENT 1: PIPELINE HAZARDS

AIM

To construct a pipeline scenario containing dependent instructions, branch instructions, and competing resource requirements, and apply pipeline stalling, operand forwarding, branch prediction, and resource duplication to minimize performance losses.

PROCEDURE

1. Define a five-stage pipeline consisting of IF, ID, EX, MEM, and WB.
2. Create a sequence of dependent and independent instructions.
3. Identify data, control, and structural hazards.
4. Insert stalls when a required operand is not available.
5. Apply operand forwarding to reduce data hazards.
6. Apply branch prediction to reduce branch delays.
7. Use resource duplication to resolve structural conflicts.
8. Calculate and compare the execution time before and after optimization.

PSEUDOCODE

START

Define pipeline stages IF, ID, EX, MEM, WB

Create instruction sequence

Identify data dependencies

Identify branch instructions

Identify resource conflicts

FOR each instruction

IF data dependency exists

 Insert pipeline stall

END IF

IF operand forwarding is possible

 Forward required operand

 Reduce stall

END IF

IF instruction is a branch

 Predict branch direction

IF prediction is incorrect

Flush incorrect instructions

END IF

END IF

IF resource conflict exists

Use duplicated hardware resource

END IF

END FOR

Calculate execution time

Compare optimized and unoptimized execution

STOP

```
Output
PIPELINE HAZARD SIMULATION
-----
I1: ADD R1,R2,R3
I2: SUB R4,R1,R5
I3: LOAD R6,0(R7)
I4: ADD R8,R6,R9
I5: BEQ R8,R0,TARGET
I6: ADD R10,R11,R12

Data Dependencies:
I2 depends on I1
I4 depends on I3
I5 depends on I4

Without Forwarding:
Stall inserted for dependent instructions
Total stalls = 3

With Operand Forwarding:
I1 -> I2 : Forwarded
I3 -> I4 : 1 stall (load-use)
I4 -> I5 : Forwarded
- - - - -
```

```
Output

With Operand Forwarding:
I1 -> I2 : Forwarded
I3 -> I4 : 1 stall (load-use)
I4 -> I5 : Forwarded
Total stalls = 1

Branch Prediction:
Prediction = NOT TAKEN
Wrong prediction penalty = 2 cycles

Resource Duplication:
Memory Unit 1 -> Instruction I3
Memory Unit 2 -> Other memory instruction
Resource conflict reduced

Performance:
Without optimization = 15 cycles
With optimization   = 11 cycles
Improvement = 26.67 %

=== Code Execution Successful ===
```

RESULT

Thus, the pipeline hazards were successfully identified and reduced using pipeline stalling, operand forwarding, branch prediction, and resource duplication, thereby improving processor performance.

EXPERIMENT 2: HIERARCHICAL BUS STRUCTURE

AIM

To develop a computer system model using a hierarchical bus structure to transfer data among the CPU, memory, and I/O devices, and analyze the effects of bus priority, bandwidth, and transfer conflicts on system performance.

PROCEDURE

1. Create the CPU, memory, and I/O devices.
2. Create a system bus and an I/O bus.
3. Connect the CPU and memory through the system bus.
4. Connect I/O devices through the I/O bus.
5. Assign priority levels to different devices.
6. Define the bandwidth of the buses.
7. Simulate data transfers between the devices.
8. Generate simultaneous bus requests to create conflicts.
9. Resolve conflicts according to device priority.
10. Analyze the effect of bandwidth and conflicts on performance.

PSEUDOCODE

START

Create CPU

Create Memory

Create I/O devices

Create System Bus

Create I/O Bus

Connect CPU and Memory to System Bus

Connect I/O devices to I/O Bus

Assign bandwidth to buses

Assign priority to devices

FOR each data transfer

 Check bus availability

 IF bus is available

 Perform data transfer

ELSE

 Check device priority

 Give bus to higher-priority device

 Lower-priority device waits

END IF

END FOR

Calculate total data transferred

Analyze waiting time and bandwidth

STOP

```
Output
HIERARCHICAL BUS SIMULATION
-----

Devices and bandwidth:
CPU = 100 MB/s
Memory = 80 MB/s
Disk = 40 MB/s
Keyboard = 10 MB/s

Bus Priority:
1 . CPU
2 . Memory
3 . Disk
4 . Keyboard

Data Transfers:
CPU -> Memory : 20 MB
Memory -> CPU : 15 MB
Disk -> Memory : 30 MB
Keyboard -> CPU : 5 MB

Total data transferred = 70 MB
```

RESULT

Thus, the hierarchical bus structure was successfully modeled for data transfer between the CPU, memory, and I/O devices. The effect of bus priority, bandwidth, and transfer conflicts on system performance was analyzed.

EXPERIMENT 3: MULTIPLE-ISSUE AND OUT-OF-ORDER PROCESSOR

AIM

To develop a processor model that supports multiple instruction issue, out-of-order execution, and speculative execution, and analyze the effects of instruction scheduling, execution units, and simultaneous multithreading on instruction throughput and resource utilization.

PROCEDURE

1. Create a set of processor instructions.
2. Define multiple execution units such as ALU, multiplier, memory unit, and branch unit.
3. Identify independent and dependent instructions.
4. Issue multiple independent instructions in the same clock cycle.
5. Schedule instructions according to dependencies and available resources.
6. Execute independent instructions out of order.
7. Predict branch instructions for speculative execution.
8. Simulate multiple threads using simultaneous multithreading.
9. Calculate instruction throughput and resource utilization.
10. Analyze the performance of the processor.

PSEUDOCODE

START

Create instruction queue

Create execution units:

ALU

Multiplier

Memory Unit

Branch Unit

WHILE instructions remain

Check instruction dependencies

Select independent instructions

FOR each selected instruction

Find available execution unit

IF execution unit is available

Issue instruction

ELSE

Wait for the resource

END IF

END FOR

Execute independent instructions out of order

IF instruction is a branch

Predict branch direction

Execute instructions speculatively

END IF

Schedule instructions from multiple threads

Update processor resources

END WHILE

Calculate total execution cycles

Calculate instruction throughput

Calculate resource utilization

STOP

Output	Output
<pre>MULTIPLE-ISSUE PROCESSOR ----- Instructions: I1 : ADD -> ALU I2 : MUL -> MUL I3 : LOAD -> MEM I4 : SUB -> ALU I5 : ADD -> ALU I6 : BRANCH -> BR Execution Units: ALU : 2 MUL : 1 MEM : 1 BR : 1 Out-of-Order Execution: Cycle 1 : I1, I2 Cycle 2 : I3, I4 Cycle 3 : I5, I6</pre>	<pre>Out-of-Order Execution: Cycle 1 : I1, I2 Cycle 2 : I3, I4 Cycle 3 : I5, I6 Speculative Execution: Processor predicts branch direction. Instructions after branch are executed speculatively. Simultaneous Multithreading: Thread 1 -> I1, I3, I5 Thread 2 -> I2, I4, I6 Performance: Instructions = 6 Cycles = 3 Throughput = 2.0 instructions/cycle Resource utilization is improved because multiple execution units work simultaneously. === Code Execution Successful ===</pre>

RESULT

Thus, the multiple-issue processor model was successfully developed. Multiple instruction issue, out-of-order execution, speculative execution, and simultaneous multithreading improved instruction throughput and processor resource utilization.

EXPERIMENT 4: PROCESSOR DATAPATH

AIM

To design and simulate a processor datapath that performs register-to-register data transfers and arithmetic operations such as increment, decrement, addition, and subtraction, and analyze the control signals and execution sequence.

PROCEDURE

1. Initialize the processor registers.
2. Connect the registers to the ALU.
3. Define the required ALU operations.
4. Define control signals for each operation.
5. Perform register-to-register data transfer.
6. Perform increment and decrement operations.
7. Perform addition and subtraction operations.
8. Store the results in the destination register.
9. Display the register values and results.
10. Analyze the control signals used for each operation.

PSEUDOCODE

START

Initialize R1 and R2

Register Transfer:

$R3 \leftarrow R1$

Increment:

$R1 \leftarrow R1 + 1$

Decrement:

$R2 \leftarrow R2 - 1$

Addition:

$R3 \leftarrow R1 + R2$

Subtraction:

$R3 \leftarrow R1 - R2$

Display register values

Display ALU operations

Display control signals

STOP

Output
<pre>PROCESSOR DATAPATH SIMULATION ----- Initial values: R1 = 10 R2 = 5 1. Register Transfer R3 <- R1 R3 = 10 2. Increment R1 <- R1 + 1 R1 = 11 3. Decrement R2 <- R2 - 1 R2 = 4 4. Addition R3 <- R1 + R2 R3 = 15 5. Subtraction R3 <- R1 - R2 R3 = 7 Control Signals: ADD -> ALU performs addition SUB -> ALU performs subtraction INC -> ALU adds 1 DEC -> ALU subtracts 1 MOV -> Register transfer === Code Execution Successful ===</pre>

RESULT

Thus, the processor datapath was successfully designed and simulated to perform register transfer, increment, decrement, addition, and subtraction operations using appropriate control signals.

EXPERIMENT 5: PIPELINED PROCESSOR PERFORMANCE

AIM

To simulate a pipelined processor executing arithmetic, memory, and branch instructions and calculate the pipeline execution time, throughput, and speedup by comparing the results with sequential instruction execution.

PROCEDURE

1. Define a five-stage pipeline consisting of IF, ID, EX, MEM, and WB.
2. Create a sequence containing arithmetic, memory, and branch instructions.
3. Calculate the sequential execution time.
4. Execute the instructions using the pipelined processor.
5. Calculate the pipeline execution time.
6. Calculate the instruction throughput.
7. Calculate the speedup obtained using pipelining.
8. Compare sequential and pipelined execution times.
9. Analyze the performance improvement.

PSEUDOCODE

START

Define pipeline stages:

IF

ID

EX

MEM

WB

Create instruction sequence

$N \leftarrow$ Number of instructions

$K \leftarrow$ Number of pipeline stages

$\text{Sequential_Time} \leftarrow N \times K$

$\text{Pipeline_Time} \leftarrow K + N - 1$

$\text{Throughput} \leftarrow N / \text{Pipeline_Time}$

$\text{Speedup} \leftarrow \text{Sequential_Time} / \text{Pipeline_Time}$

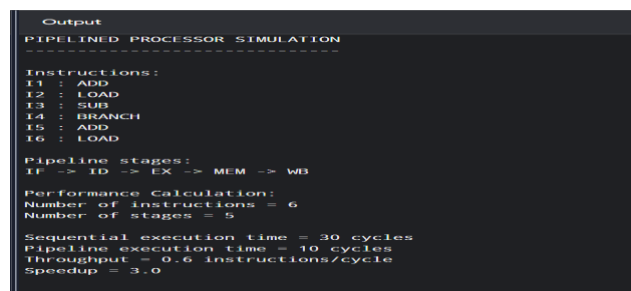
Display Sequential_Time

Display Pipeline_Time

Display Throughput

Display Speedup

STOP



```
Output
PIPELINED PROCESSOR SIMULATION
-----
Instructions:
I1 : ADD
I2 : LOAD
I3 : SUB
I4 : BRANCH
I5 : ADD
I6 : LOAD

Pipeline stages:
IF -> ID -> EX -> MEM -> WB

Performance Calculation:
Number of instructions = 6
Number of stages = 5

Sequential execution time = 30 cycles
Pipeline execution time = 10 cycles
Throughput = 0.6 instructions/cycle
Speedup = 3.0
```

RESULT

Thus, the pipelined processor was successfully simulated. The pipeline execution time, throughput, and speedup were calculated and compared with sequential execution, demonstrating the performance improvement achieved through pipelining.

